

Documentation

Technique

<https://github.com/TonyLita/Cloud/tree/projet>

Système de Gestion et Suivi des
Incidents Routiers

1. Présentation du Projet

Ce projet est une solution complète multi-plateforme pour le signalement et le suivi des dégradations routières. Il combine une application mobile pour les citoyens (signalement offline/online) et une interface web d'administration pour la gestion des travaux.

Architecture Technique :

- **Backend** **Spring Boot 3** (Java 17+) + PostgreSQL 15
- **Web Admin** **Vue.js 3** (Vite)
- **Mobile** **Ionic 8 + React**
- **Infrastructure** **Docker Compose** (Orchestration complète)
- **Cartographie** : Leaflet + Tileservier GL (Cartes hors-ligne)
- **Synchronisation** : Algorithme hybride PostgreSQL / Firebase Firestore

2. Prérequis

Avant de démarrer le projet, assurez-vous d'avoir installé les outils suivants :

- **Docker Desktop** (Recommandé pour un démarrage rapide)
- **Node.js 18+ & Yarn** (Pour le développement Frontend)
- **Java JDK 17+ & Maven** (Pour le développement Backend)

3. Installation et Démarrage (Méthode Rapide)

La méthode recommandée pour tester l'application est d'utiliser Docker Compose. Cela lancera tous les modules (Base de données, Backend, Front Web, Mobile Web, Serveur de Tuiles).

Note : Assurez-vous que les ports 8080 , 5432 , 80 , 81 et 8085 sont libres.

Commandes :

```
# 1. Cloner le projet (si ce n'est pas fait)
git clone -b projet https://github.com/TonyLita/Cloud.git
cd Cloud

# 2. Lancer l'environnement complet
docker-compose up --build

# Pour arrêter : Ctrl+C ou "docker-compose down"
```

Accès aux Applications :

- **Web Admin** : <http://localhost:80>
- **App Mobile (Version Web)** : <http://localhost:81>
- **API Backend** : <http://localhost:8080/api>
- **Documentation API (Swagger)** : <http://localhost:8080/swagger-ui/index.html>
- **Serveur de Tuiles (Cartes)** : <http://localhost:8085>

4. Installation Manuelle (Développement)

A. Backend (Spring Boot)

Configuration : Modifier

Backend/src/main/resources/application.properties si nécessaire (pour la BDD locale).

```
cd Backend
# Compiler et packager
mvn clean package -DskipTests
```

```
# Lancer
java -jar target/cloud-0.0.1-SNAPSHOT.jar
```

B. Web Admin (Vue.js)

```
cd web
# Installer les dépendances
yarn install

# Lancer en mode développement (Hot-reload)
yarn serve
```

Accès : <http://localhost:8081> (ou port indiqué)

C. Mobile (Ionic/React)

```
cd mobile
# Installer les dépendances
npm install

# Lancer le serveur de développement
npm run dev
```

Accès : <http://localhost:5173>

Note : Pour générer l'APK Android :

```
npx cap sync android
npx cap open android
```

5. Fonctionnalités Clés

- **Signalement Géolocalisé :** Utilisation du GPS et placement manuel sur carte.
- **Mode Hors-Ligne :** Les cartes sont servies localement via Tilesserver GL (.mbtiles).
- **Synchronisation Bidirectionnelle :** Les données sont stockées localement (Postgres/SQLite Mobile) et synchronisées avec Firebase dès qu'une connexion internet est détectée.

- **Gestion Administrative** : Tableau de bord pour valider les signalements, gérer les entreprises de BTP, et suivre l'avancement des travaux.
-